Module 2: Using Git Locally

Section 1: Advanced Git interaction

- This lecture introduces the upcoming advanced features of Git, focusing on enhancing your version control skills beyond the basics.

Advanced Git Features Overview

- The course will cover useful shortcuts and ways to extract more information from Git.
- It emphasizes the power of undoing changes, including discarding file changes, fixing incorrect commits, and rolling back to previous project snapshots.

Undoing Changes Techniques

- Various techniques will be explored to undo changes depending on the situation.
- Understanding when and how to use each undo method is a key learning objective.

Branching and Collaboration

- The concept of branches will be introduced to allow working on experimental features without affecting the main code.
- The course will explain how to use branches, handle merge conflicts, and support multiple versions of a project.

--Check **"READING_01"** file--

- This lecture explains how to use the git commit command with the -a flag to streamline the commit process by skipping the staging step for tracked files.

Using git commit -a to Skip Staging

- The -a flag automatically stages all modified tracked files and commits them in one step, bypassing the need for a separate git add command.
- This shortcut does not work for new untracked files, which still require git add to be tracked before committing.

Practical Example and Usage Tips

- The example modifies a script, then commits changes using git commit -a with a message via the -m flag for a quick commit.
- The -a shortcut is best for small, straightforward changes where detailed commit messages and staging control are not necessary.

Understanding HEAD in Git

- HEAD is a pointer representing the currently checked out snapshot of the project, indicating the current state of the working directory.
- It acts like a bookmark to track your position in the project history and is used by Git commands to reference the current commit or branch.

--Check **"READING_02"** file--

- This lecture explains how to use Git commands to view and manage changes in a repository, focusing on inspecting commit history and tracking uncommitted changes.

Using Git Log and Show to Inspect Commits

- The `git log` command lists commits with messages, authors, and dates; adding the `-p` flag shows the patch (line-by-line changes) for each commit.
- The `git show` command displays details and patches for a specific commit by its ID, useful for quickly examining particular changes.

Reviewing Change Statistics and Uncommitted Changes

- The `git log --stat` flag provides a summary of files changed and lines added or removed in each commit.
- The `git diff` command shows differences between the working directory and the staging area, helping track uncommitted changes.

Staging Changes and Preparing Commits

- Using `git add -p` allows reviewing and selectively staging changes interactively before committing.
- After staging, `git diff --staged` shows the changes ready to be committed, ensuring accuracy before running `git commit`.

--Check **"READING_03"** file--

- This lecture explains how to manage files in a Git repository, including removing, renaming, and ignoring files.

Removing and Renaming Files

- Use the git rm command to remove files from the repository; this deletes the file and stages the removal for the next commit.

- Use the git mv command to rename or move files within the repository; this stages the change but requires a commit to finalize it.

Tracking Changes and Status

- The git status command shows the current state of files, including added, modified, deleted, or renamed files, helping you keep track of changes.
- After staging changes like removals or renames, you must commit them with a descriptive message to record the update in the repository history.

Ignoring Unwanted Files

- To prevent certain files (e.g., system-generated files like .DS_Store) from cluttering the status output, create a .gitignore file listing these files.
- The .gitignore file itself must be added and committed to the repository to take effect and keep the repository clean from unwanted files.

--Check **"Study_Guide_01"** file--

Section 2: Undoing things

--Check **"READING_04"** file--

- This course content explains how to undo changes in Git, a version control system, using different commands depending on the state of the changes.

Undoing Unstaged Changes

- Use `git checkout <file>` to revert a file back to its last committed or staged state, discarding any modifications made since.
- The `-p` flag with `git checkout` allows selectively undoing changes chunk by chunk.

Undoing Staged Changes

- If changes have been added to the staging area but not yet committed, use `git reset HEAD <file>` to unstage them.
- The `-p` flag with `git reset HEAD` lets you selectively unstage specific changes.

--Check **"READING_05"** file--

- This lecture explains how to fix mistakes in Git commits using the --amend option.

Fixing Commit Content

- If you forget to add a file to a recent commit, you can stage the missing file and run `git commit --amend` to update the commit with the new changes.
- The amend command opens an editor allowing you to modify the commit message and include all intended files in the commit.

Improving Commit Messages

- You can also use `git commit --amend` to improve or clarify the commit message without changing the staged files.
- This helps keep commit history clear and descriptive, linking to relevant issues or explaining the purpose of changes.

Important Considerations

- Amending commits rewrites history by replacing the previous commit with the amended one.
- Avoid using --amend on commits that have already been pushed to shared or public repositories to prevent confusion in collaborative workflows.

--Check **"READING_06"** file--

- This lecture explains how to handle and undo problematic commits in Git using the git revert command.

Understanding git revert

- git revert creates a new commit that undoes the changes made in a previous commit by applying the inverse of those changes.
- This approach preserves the project's commit history, showing both the original and the rollback commits.

Using git revert in practice

- You can revert the latest commit by running git revert HEAD, which targets the current commit snapshot.
- After running git revert, Git opens a commit message editor where you can explain why the rollback is necessary for future reference.

Reviewing the effects of revert

- The revert commit removes the changes introduced by the faulty commit, as seen in the commit log with added lines marked by plus signs and removed lines by minus signs.

- This method allows you to fix errors quickly while maintaining a clear and consistent project history.

--Check **“READING_07”** file--

- This lecture explains how Git uses commit IDs to identify specific commits in the version history, enabling targeted rollbacks and ensuring data consistency.

Commit IDs and Their Purpose

- Commit IDs are 40-character strings generated by the SHA1 hash algorithm, representing all data related to a commit.
- These hashes guarantee repository consistency by ensuring the data retrieved matches exactly what was committed, helping detect corruption or tampering.

How Git Uses Commit IDs

- Git uses commit IDs instead of simple counters to uniquely identify commits, preventing collisions and maintaining data integrity.
- Changing any commit data, such as amending a commit, results in a new commit ID, which is why amending public commits is discouraged.

Working with Specific Commits

- You can specify a commit by its full or partial commit ID (usually 4-8 characters) to view or revert it.
- The git revert command creates a new commit that undoes changes from a specified earlier commit, including the original commit ID in the revert message for clarity.

--Check **“Study_Guide_02”** file--

Section 3: Branching and Merging

- This lecture explains the concept and importance of branches in Git, a version control system.

What is a Branch in Git

- A branch is essentially a pointer to a specific commit, representing an independent line of development in a project.
- The default branch created in a new Git repository is called "master," which usually holds the stable, known good state of the project.

Purpose and Use of Branches

- Branches allow developers to work on new features or experiments separately without affecting the stable master branch.
- This separation helps in safely trying out new ideas, fixing bugs, or adding features before merging them back into the master branch.

Branching Workflow Example

- Instead of directly committing potentially problematic changes to master, developers create a new branch (e.g., "add disk full") to develop and test changes.
- Once the changes are verified, they can be merged back into master, or discarded if they don't work out, without impacting the main project.

--Check **"READING_08"** file--

- This lecture explains how to work with branches in Git, focusing on creating, listing, switching, and committing changes to branches.

Using Git Branch Commands

- The `git branch` command lists all branches in a repository and can also create new branches by specifying a branch name.
- The current branch is indicated with an asterisk (*) in the branch list output.

Switching Between Branches

- The `git checkout` command is used to switch between branches, updating the working directory to match the selected branch.
- A shortcut `git checkout -b <branch-name>` creates a new branch and switches to it immediately.

Committing Changes on Branches

- After switching to a new branch, you can create and commit new files specific to that branch.
- Git shows the relationship between branches and commits, indicating which branch is ahead or behind based on commit history.

--Check **"READING_09"** file--

- This lecture explains how to work with branches in Git, focusing on creating, switching, and deleting branches.

Branch Creation and Switching

- Creating a new branch allows you to work on changes separately from the master branch.
- Switching branches with git checkout changes the HEAD pointer and updates the working directory to reflect the snapshot of the selected branch.

Branch History and Working Directory

- Each branch points to a specific commit, and switching branches changes the commit history and files visible in the working directory.
- Changes committed on one branch do not appear in another branch unless merged.

Deleting Branches

- You can delete a branch using git branch -d if it is no longer needed.
- Git prevents deleting branches with unmerged changes unless forced, encouraging you to merge changes first to avoid data loss.

--Check **"READING_10"** file--

- This lecture explains how to manage branches and perform merges in Git, a version control system.

Branch Management and Merging

- Developers create separate branches to develop new features or changes independently from the main codebase.
- Once a feature is ready, the separate branch is merged back into the main branch (often called master).

Types of Git Merges

- Fast Forward Merge: Happens when the branch being merged has all its commits already in the current branch, so Git just moves the pointer forward without creating a new commit.
- Three-Way Merge: Occurs when the branches have diverged, meaning both have unique commits after branching. Git creates a new merge commit combining the histories.

Handling Merge Conflicts

- Git tries to automatically combine changes if they affect different files or different parts of the same file.
- If changes conflict in the same part of a file, Git cannot merge automatically, resulting in a merge conflict that requires manual resolution.

--Check **“READING_11”** file--

- This lecture explains how to handle merge conflicts in Git when two branches have conflicting changes in the same file.

Understanding Merge Conflicts

- Merge conflicts occur when Git cannot automatically merge changes from two branches because they edit the same part of a file.
- Git marks the conflicting sections in the file to help the user decide how to resolve them.

Resolving Conflicts

- Users must manually edit the conflicted file to keep, modify, or combine changes from both branches.
- After resolving, the user marks the conflict as resolved by staging the file with git add and then commits the merge.

Additional Tips and Commands

- The git status command provides information about unmerged files and conflict resolution status.
- If needed, git merge --abort can be used to cancel the merge and revert to the previous state before the merge started.

--Check **“Study_Guide_03”** file--

Section 4: Module 2 Review

- This lecture summarizes key Git concepts covered in recent lessons, focusing on advanced Git commands, undoing changes, and branching.

Advanced Git Commands

- Learned how to skip the staging area, obtain detailed commit information, and manage files by deleting or renaming them within a repository.
- Explored commands that enhance interaction with Git beyond basic usage.

Undoing Changes

- Covered methods to revert both unstaged and staged changes.

- Learned how to amend commits and perform rollbacks from the latest or older commits to correct mistakes.

Branching and Merging

- Understood how to create, switch between, and delete branches.
- Explored merging branches and resolving merge conflicts to manage parallel development effectively.

--Check **“Glossary”** file--

--Check **“Exemplar”** file--